

Wireless Optical Mouse

PRX firmware — architecture, USB HID interface and validation

nRF52840 PCA10059 | Enhanced ShockBurst | USB HID | Remote Wake

Project Overview

The PRX firmware runs on a Nordic Semiconductor nRF52840 PCA10059 USB dongle and forms the receiver side of the wireless optical mouse. It receives mouse packets from the nRF52833 PTX over a 2.4 GHz Enhanced ShockBurst (ESB) link, accumulates received relative motion and button state, and presents that input to the host as a standard USB HID mouse.

The radio receive path and USB report path are intentionally decoupled. ESB packets can arrive between USB report opportunities; their X/Y deltas are accumulated in software until the USB endpoint is ready to transmit a HID report. This allows the PRX to bridge an asynchronous wireless input stream to the host USB schedule without requiring the two interfaces to run in lockstep.

Hardware

nRF52840 PCA10059 USB Receiver

The receiver is implemented on the nRF52840 PCA10059 USB dongle. The nRF52840 provides the 2.4 GHz radio for ESB communication and the USB device peripheral used to enumerate as a HID mouse.

The dongle is connected directly to the host PC through USB. Its firmware receives the PTX application packet, tracks pending mouse state, constructs HID input reports, and transmits those reports through USB interrupt endpoint IN1.

Architecture and Data Flow

The complete input path is:

PMW3389 / buttons → nRF52833 PTX → ESB → nRF52840 PRX → USB HID → host PC

PRX Data Path

When an ESB RX event occurs, the receiver drains available payloads from the ESB RX FIFO. Payloads are accepted only when their length matches the 6-byte mouse packet structure. The packet contains a diagnostic sequence byte, the current button bit field, and signed 16-bit X/Y motion deltas.

Accepted X/Y deltas are added to signed 32-bit pending accumulators. Button-state changes update the stored current button value and set a pending-change flag. A transition containing a newly pressed button also raises a remote-wakeup request flag for use if the USB host is suspended.

At a USB Start-of-Frame opportunity, if endpoint IN1 is idle and no report is already queued, the PRX snapshots the current pending state. X/Y values are clamped to signed 16-bit HID range and combined with the current button state to form the next HID report.

The report snapshot is not removed from the PRX pending state when it is queued. It is committed only after endpoint IN1 reports a successful transfer. If additional ESB packets arrive while the USB report is in flight, their motion remains in the accumulators. This prevents newly received motion from being removed when the earlier USB snapshot completes.

Timing Relationship

USB endpoint IN1 is described with a 1 ms polling interval. ESB reception is asynchronous with respect to that interval, so multiple wireless packets may contribute to one USB report. The 1 ms interval therefore defines USB report opportunities rather than the arrival rate of wireless mouse packets.

Interface / event	Behavior
ESB RX	Asynchronous packet reception; valid motion is accumulated immediately
USB SOF	Opportunity to snapshot pending input and queue a HID report
EPIN1 success	Commit the exact report snapshot that was delivered to the host
USB suspend	Stop normal SOF-driven report generation and enter USB suspend state
Remote wake request	Generated from a new button press when host remote wake is enabled

Wireless Receive Interface

ESB Mouse Packet

The PRX receives the same fixed 6-byte application payload transmitted by the PTX:

Byte offset	Field	Type	Description
0	packet_sequence	uint8_t	Diagnostic sequence value
1	buttons	uint8_t	Current mouse-button bit field
2–3	x	int16_t	Signed relative X-axis motion
4–5	y	int16_t	Signed relative Y-axis motion

```
typedef struct __attribute__((packed))
```

```
{
    uint8_t packet_sequence;
    uint8_t buttons;
    int16_t x;
    int16_t y;
} mouse_packet_t;
```

A compile-time size assertion verifies that the packet structure remains exactly 6 bytes. The PRX also checks the received payload length before interpreting the payload as a mouse packet.

The sequence byte is retained primarily as a diagnostic field. Normal receiver operation does not depend on it, but it can be used later to correlate PTX and PRX logs or identify gaps in received application packets.

Receive Accumulation

The PRX maintains signed 32-bit pending X/Y accumulators. Every valid received packet adds its motion to these totals. When USB needs a report, the pending values are copied and clamped to signed 16-bit range rather than immediately removed.

This design allows several ESB packets to be combined into one HID report and also preserves motion that exceeds the range of a single HID report across later USB transfers.

Button State

The buttons field represents the current button state. If a received packet changes that state, the PRX records the new value and marks the button state as pending for USB delivery. The pending-change flag is cleared after a successful USB report only if the transmitted button snapshot still matches the current state; a newer button transition therefore remains pending.

USB HID Interface

HID Report Format

The PRX presents a packed 6-byte mouse input report to the USB host:

Byte offset	Field	Type	Description
0	buttons	uint8_t	Eight button bits; current prototype uses left/right
1–2	x	int16_t	Signed relative X-axis motion
3–4	y	int16_t	Signed relative Y-axis motion
5	wheel	int8_t	Relative wheel value; currently sent as zero

```
typedef struct __attribute__((packed))
```

```
{
    uint8_t buttons;
    int16_t x;
    int16_t y;
    int8_t wheel;
} hid_mouse_report_t;
```

The HID report descriptor declares eight one-bit buttons, signed 16-bit relative X and Y axes, and an 8-bit relative wheel. The report structure is statically asserted to remain 6 bytes.

USB Report Scheduling

The endpoint descriptor uses interrupt endpoint IN1 with a 1 ms polling interval. On each USB Start-of-Frame event, the firmware checks whether the endpoint and transmit buffer are available. If pending mouse input exists, it snapshots that state and queues a HID report.

The queued report is associated with a saved PRX-side snapshot. When the IN1 transfer completes successfully, the firmware subtracts that exact X/Y snapshot from the pending accumulators and conditionally clears the button-change flag. If the transfer fails or is aborted, the motion is not committed, so it remains available for a later report.

Why the Radio and USB Paths Are Decoupled

ESB can deliver input updates more frequently or at different times than the USB host requests reports. Accumulating radio input on the PRX and committing it only after successful USB delivery prevents the receiver from requiring one radio packet per USB frame and avoids discarding input when endpoint IN1 is temporarily unavailable.

USB Suspend and Remote Wake

Suspend / Resume State

The USB event handler tracks whether the device is configured and whether the bus is suspended. On USB suspend, the firmware marks the device suspended and calls the nRF52 USB suspend function. Resume and USB wake-request events clear the suspended state. USB reset or power removal also clears the stored suspend and remote-wakeup-enable state.

Host Remote-Wakeup Enable

Remote wake is host-controlled. The firmware tracks the standard USB SetFeature/ClearFeature state for device remote wake and issues a wake request only when that feature has been enabled by the host.

Button-Triggered Wake Path

A newly pressed button is detected in the ESB receive path by comparing the incoming button field with the previously stored state. The receive handler sets a pending wake flag when a new press is observed. The main loop consumes that flag independently of USB SOF processing.

A USB remote-wakeup request is issued only when all of the following are true:

- a new button press has requested wakeup;
- the USB device is configured;
- the USB bus is currently suspended; and
- the host has enabled remote wakeup.

Keeping remote-wake processing outside the SOF-driven report path is important because normal SOF traffic is absent while the bus is suspended.

Observed Remote-Wake Behavior

System-level testing on a host that maintained USB power during suspend confirmed that a mouse-button press could wake the suspended host through the PRX USB interface. The test was repeated successfully across more than one suspend/wake cycle. This validates the implemented wake path functionally; wake latency and host-specific power behavior were not measured quantitatively.

Firmware Modules

main.c — Application Control

The PRX main function initializes platform support, status LEDs, the ESB receiver, and the USB mouse interface. The main loop services pending remote-wakeup requests, processes logging, and otherwise allows interrupt-driven ESB and USB activity to perform the data-path work.

esb_mouse.c / esb_mouse.h — Wireless Receive and Pending Input

The ESB module configures the receiver and owns the wireless mouse packet format. Its RX event handler validates packet size, updates the pending X/Y accumulators, tracks current button state, and raises button-triggered remote-wakeup requests.

The module exposes peek/commit functions to the USB layer. The peek operation snapshots pending input without removing it; the commit operation subtracts the successfully delivered report from the accumulator and clears button-change state only when appropriate.

usb_mouse.c / usb_mouse.h — USB HID Device

The USB module defines the device, configuration, HID, endpoint, and report descriptors, manages USB power and bus events, builds the packed 6-byte HID mouse report, and queues IN1 transfers. It also owns suspend/resume tracking and the call that requests host remote wake.

Key Design Choices

Choice	Reason
32-bit PRX motion accumulators	Preserve multiple received deltas between USB reports and allow partial draining through 16-bit HID reports.
Peek then commit on EPIN1 success	Do not discard input merely because a report was queued; preserve later arrivals while the report is in flight.
1 ms USB interrupt interval	Provide a 1 kHz USB report opportunity consistent with the intended mouse interface.
Remote wake from button edge	A discrete button press provides an unambiguous wake event while USB SOF is stopped during suspend.
Sequence byte retained	Low-cost diagnostic aid; not required by the normal input path.

Testing and Results

PRX validation focused on functional correctness of the receive-to-USB path and suspend/wake behavior. Quantitative USB throughput, end-to-end latency, current consumption, and controlled RF reliability were not measured.

Functional Input

End-to-end testing with the PTX and host PC confirmed normal cursor movement and mouse-button operation through the PRX. The receiver accepted the updated 6-byte PTX packet format and generated the corresponding 6-byte USB HID reports with signed 16-bit X/Y motion.

Motion Preservation Across USB Transfers

The PRX firmware was changed from immediate radio-to-USB forwarding to a pending-input model. Received motion is accumulated, a USB snapshot is created only when the endpoint is ready, and that snapshot is committed only after successful IN1 completion. This design specifically addresses the case where additional radio packets arrive while a USB transfer is already in flight.

USB Suspend and Remote Wake

Testing confirmed USB suspend detection and successful button-triggered remote wake. The host resumed and the mouse continued operating after wake. The wake behavior was also observed on a subsequent suspend/wake cycle, providing a repeatability check for the implemented path.

Validation Scope

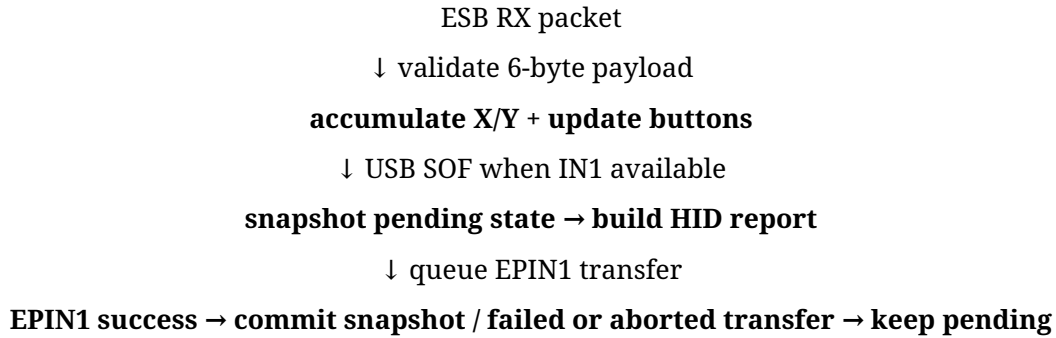
Source-level review confirms the pending-input and commit-on-success implementation. Functional testing confirmed end-to-end input, basic button handling, USB suspend/resume behavior, and remote wake on a host that maintained USB power during suspend. The current tests do not establish maximum sustainable ESB receive rate, USB report throughput under controlled load, end-to-end input latency, exact wake latency, power consumption, or behavior across multiple host operating systems.

System and Firmware Diagrams

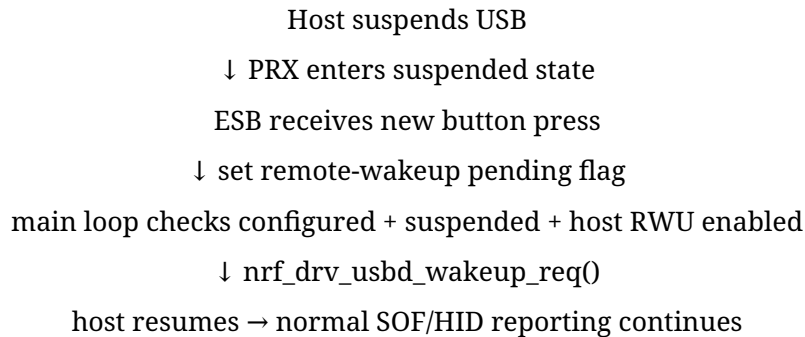
System Overview

Stage	Flow
Input	PMW3389 sensor + PTX buttons
Wireless TX	nRF52833 PTX
Wireless RX	nRF52840 PRX
Host interface	USB HID interrupt IN
Destination	Host PC

PRX Data Flow



Suspend / Remote-Wake Flow



Implementation References

The implementation is based on the Nordic nRF5 SDK ESB and USB device drivers used by the project. The technical statements in this document reflect the current project source and the functional tests performed during development.